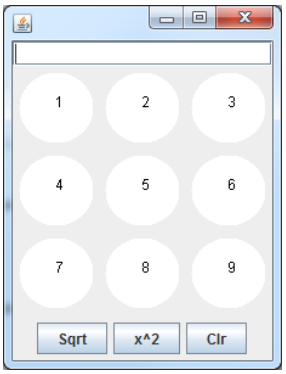
# COMP167 Lab – Event Handling

**Description:**




In this lab exercise, you will create a simple JavaFX calculator. The GUI will have a pane of graphic buttons (filled ovals) with string labels. When pressed, the graphic button background will change color indicating the button is currently pressed. When the mouse is released, the background will return to its original color. Also, the string value in the button will be concatenated to the TextField at the top of the GUI. There will be additional command Buttons at the bottom of the GUI with functionality described below.

**CircleButton Class (extends StackPane):**

The graphic circular buttons are created by drawing a filled circle on a StackPane. So, the pictured GUI uses 9 different StackPanes for displaying the 9 graphic buttons. Of course, these CircleButton objects can then be placed on a single GridPane to achieve the 3x3 layout (see SimpleCalc class below). Create a class named CircleButton that extends the StackPane class. The class should have a Label property and a Circle property. By default, the background of the circle should be white and the text black. The no-arg constructor sets the colors to their defaults and the text to the empty string "". A second constructor should have a String parameter to initialize the Label property. Set the preferred size of the CircleButton pane to 100 but the radius of the circle to 45 pixels. Put this class in a separate file.

| CircleButton | |
|---|---|
| -lblValue : Label<br>-circle : Circle | Displays button numeric value<br>Circle object that is displayed. The radius should be 45 pixels |
| +CircleButton()<br>+CircleButton( val : String )<br>+setColor( Color color ) : void<br>+getColor() : Color<br>+getValue() : String | Create a button with no displayed value and a white background.<br>Create a button with val as the numeric label and a white background.<br>Set the background/fill color of the CircleButton.<br>Get the current fill color of the CircleButton.<br>Get the value currently displayed on the CircleButton. |

**CalcPane Class (extends BorderPane):**

This class will extend a BorderPane and will contain a TextField at the top, a GridPane of 9 CircleButtons in the center and an HBox of three Buttons at the bottom. When adding the CircleButtons, you should use nested for-loops to facilitate the instantiation of the CircleButtons with the correct text label (1..9 ) and the registration of a single Listener for all nine CircleButtons.

| CalcPane | |
|---|---|
| -buttonPane:GridPane<br>-functionPane:HBox<br>-textDisplay:TextField<br>-btnSquareRoot:Button<br>-btnSquare:Button<br>-btnClear:Button | <br><br>Make sure you import javafx.scene.control.TextField and NOT AWT version<br>Make sure you import javafx.scene.control.Button and NOT AWT version |
| | |

**Mouse Events:**

Create an inner class that implements the EventHandler<MouseEvent> interface. Instantiate an object of the inner class and register it with all nine CircleButtons. The handle() method should toggle the clicked CircleButton background to a new color using the *setFill()* method. In addition, the numeric value on the CircleButton should be appended to the current value in the top TextField. On mouse release, change the background back to the original color. You must call the methods for registering mouse pressed and mouse released for each CircleButton.

Algorithm for the handle() method registered with the CirleButtons:

- Use the event object passed to handle() to get a reference (e.getSource()) to the CircleButton that was clicked.
- Check the current fill color of the CircleButton
    - If the fill color is white, this is a mouse press event so change the fill color to YELLOW and append the value on the button to the current TextField value (mouse pressed).
    - Else change the fill color back to white (mouse released).

**Command Buttons:**

The Buttons in the bottom HBox pane will perform the following functions on the current value in the TextField:

- x^2 – Replace the current TextField with the square of the current value in the TextField
- sqrt - Replace the current TextField with the square root of the current value in the TextField
- Clear - Clear the contents of the TextField

You can create another inner class that implements the EventHandler interface to handle the button click events or you can create anonymous listeners for the Buttons.

**SimpleCalc Class:**

This class will extend the Application class and contain your main() method. Since it extends Application, it will override the start() method. In the start() method you will instantiate a CalcPane, add it to a new Scene, and add the other code that is expected for a JavaFX application.

**Step 1:**

Create your new lab project using the GitHub project sources. Go to Project Structure in the File menu and select Libraries. Add a new library by clicking the "+" button and navigating to the Lib directory of your JavaFX SDK. If the project creation step created a Main.java file, delete it. Modify the SimpleCalc class in the src folder so that it extends the Application class. Inside the class body, add a stub for the start method followed by the main method (just contains the `launch( args );` statement). Select run menu and then select Edit Configuration. Add a new run configuration. Select SimpleCalc as the class with your main() and then select Modify options->Add VM options. Add the following:

--module-path "*path-to-your-javaFX-lib-directory*" --add-modules javafx.controls

Replace *path-to-your-javaFX-lib-directory* with the path to the JavaFX lib directory on your computer.

**Step 2:**

Create the CircleButton class as described above. The constructors should instantiate Label and Circle objects. Use the setStroke() and setFill() methods to set the color of the circle outline and interior color. Add both objects to the StackPane: *this.getChildren().addAll( circle, lblValue)*

**Step 3:**

Test your code now before moving on. Go to your SimpleCalc class and add code to your start method to instantiate a new CircleButton class with a parameter of "Hello". Set the size of the CircleButton to 100x100: setPrefSize(100,100). Create a new Scene, add the CircleBotton, and then add the Scene to the stage. Now you can show() the stage. Now run the application and the CircleButton should appear.

**Step 4:**

Create the CalcPane class as an extended BorderPane. The constructor will have nested for-loops similar to what was done in the TicTacToe example from lecture. Instantiate the GridPane before the for-loops. Instantiate each CircleButton with a label from "1" to "9" by using a separate counter variable that is incremented after each CircleButton instantiation and add the CircleButton to the GridPane using the loop control variables to specify col, row in the GridPane. Remember, the column comes first. Add the GridPane to the Center zone of the BorderPane. Instantiate the TextField and add it to the Top zone of the BorderPane. Create a HBox (functionPane), instantiate the three buttons and add them to the HBox. Add the HBox to the Bottom zone of the BorderPane.

**Step 5:**

Test your code by creating a CalcPane in the start() method. Now, add the CalcPane to the Scene instead of the CircleButton. Your app should now show the entire GUI.

**Step 6:**

Let's add the event handling to your code. In your CalcPane class, create an innerclass named CircleButtonHandler that implements the EventHandler<MouseEvent> interface. The handle() method should implement the algorithm described earlier in the document. Add code to create a CircleButtonHandler object before the nested for-loops. Register the listener with each CircleButton as it is instantiated using the setOnMousePressed( handlerObjectName ) and setOnMouseReleased(handlerObjectName) methods.

For the function buttons in the bottom zone. Here is some short-cut code called lambda expressions to handle the button clicks. Add this code after you have instantiated the buttons:

```
btnSquareRoot.setOnAction(e -> { textDisplay.setText(Math.sqrt(Double.parseDouble(textDisplay.getText()))+""); });
btnSquare.setOnAction(e -> { textDisplay.setText(Math.pow(Double.parseDouble(textDisplay.getText()),2)+""); });
btnClear.setOnAction(e -> {textDisplay.setText("");});
```

Note: Some helpful functions are setSpacing() for the HBox, setMinWidth() for the function Buttons (I used a width of 75), setAlignment( Pos.Center ) for the HBox, setPrefSize( 100, 100 ) for the CircleButtons after they are instantiated.

**Step 7:**

Test your final code.

**Grading:**

Steps 1 – 3: 4 points

Steps 4 – 5: 7points  (for a total of 11 points)

Steps 6-7: 9 points (for a total of 20 points)